Artificial Intelligence Technology and Application

Deep Learning Lab Guide

Student Version

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD.

About This Document

Description

This guide introduces the following five exercises:

- Exercise 1: MindSpore basics, which describes the basic syntax and common modules of MindSpore.
- Exercise 2: Handwritten character recognition, in which the MindSpore framework is used to recognize handwritten characters.
- Exercise 3: MobileNetV2 image classification, which mainly introduces the classification of flower images using the lightweight network MobileNetV2.
- Exercise 4: ResNet-50 image classification exercise, which mainly introduces the classification of flower images using the ResNet-50 model.
- Exercise 5: TextCNN sentiment analysis, which mainly introduces sentiment analysis of statements using the TextCNN model.

Background Knowledge Required

This course is for Huawei's basic certification. To better understand this course, familiarize yourself with the following:

- Basic knowledge of Python, MindSpore concepts, and Python programming.

Contents

About This Document 1
1 TextCNN Sentiment Analysis 3
1.1 Introduction 3
1.2 Preparations 3
1.3 Detailed Design and Implementation 3
1.3.1 Data Preparation 3
1.3.2 Procedure 4
1.4 Question 14

1 TextCNN Sentiment Analysis

1.1 Introduction

This exercise implements the TextCNN sentiment analysis, a classic case in the deep learning field. The entire process is as follows:

- Download the required dataset. (The rt-polarity dataset is used to define the data preprocessing function.)
- Generate data for training and validation.
- Define the TextCNN model structure build, training, validation, offline model loading, and online inference functions.
- Define various parameters required for training, such as optimizer, loss function, checkpoint, and time monitor.
- Load the dataset and perform training. After the training is complete, use the test set for validation.

1.2 Preparations

Before you start, check whether MindSpore has been correctly installed. You are advised to install MindSpore on your computer by referring to the MindSpore official website https://www.mindspore.cn/install/en.

In addition, you should have basic mathematical knowledge, including basic knowledge of Python coding basics, probability, and matrices.

Recommended environment:

Version: MindSpore 1.7

Programming language: Python 3.7

1.3 Detailed Design and Implementation

1.3.1 Data Preparation

The rt-polarity used in this example contains English movie reviews. The **rt-polarity.pos** file stores 5000 positive reviews, and the **rt-polarity.neg** file stores 5000 negative reviews.

Download link: https://certification-data.obs.cn-north-4.myhuaweicloud.com/ENG/HCIA-AI/V3.5/chapter4/TextCNN.zip

The data directory structure is as follows:

```
└─data
        ├─ rt-polarity.pos
        └─ rt-polarity.neg
```

1.3.2 Procedure

Step 1 Import the Python library and module and configure running information.

Import the required Python library.

Currently, the **os** library is required. Other required libraries will not be described here. For details about the MindSpore modules, see the MindSpore API page. You can use context.set_context to configure the information required for running, such as the running mode, backend information, and hardware information.

Import the context module and configure the required information.

Code:

```
import math
import numpy as np
import pandas as pd
import os
import math
import random
import codecs
from pathlib import Path

import mindspore
import mindspore.dataset as ds
import mindspore.nn as nn
from mindspore import Tensor
from mindspore import context
from mindspore.train.model import Model
from mindspore.nn.metrics import Accuracy
from mindspore.train.serialization import load_checkpoint, load_param_into_net
from mindspore.train.callback import ModelCheckpoint, CheckpointConfig, LossMonitor, TimeMonitor
from mindspore.ops import operations as ops

from easydict import EasyDict as edict


cfg = edict({
    'name': 'movie review',
    'pre_trained': False,
    'num_classes': 2,
    'batch_size': 64,
    'epoch_size': 4,
    'weight_decay': 3e-5,
    'data_path': './data/',
    'device_target': 'CPU',
    'device_id': 0,
    'keep_checkpoint_max': 1,
```

```
    'checkpoint_path': './ckpt/train_textcnn-4_149.ckpt',
    'word_len': 51,
    'vec_length': 40
})

context.set_context(mode=context.GRAPH_MODE, device_target=cfg.device_target,
device_id=cfg.device_id)
```

The graph mode is used in this exercise. You can configure hardware information as required. For example, if the code runs on the Ascend AI processor, set **device_target** to **Ascend**. This rule also applies to the code running on the CPU and GPU. For details about the parameters, see the context.set_context API description at https://www.mindspore.cn/docs/en/r1.7/api_python/mindspore.context.html.

Step 2 Read and process data.

Data preview code:

```
with open("./data/rt-polarity.neg", 'r', encoding='utf-8') as f:
        print("Negative reivews:")
        for i in range(5):
            print("[{0}]:{1}".format(i,f.readline()))
with open("./data/rt-polarity.pos", 'r', encoding='utf-8') as f:
        print("Positive reivews:")
        for i in range(5):
            print("[{0}]:{1}".format(i,f.readline()))
```

Data preview output:

```
Negative reivews:
[0]:simplistic , silly and tedious .

[1]:it's so laddish and juvenile , only teenage boys could possibly find it funny .

[2]:exploitative and largely devoid of the depth or sophistication that would make watching such a graphic treatment of the crimes bearable .

[3]:[garbus] discards the potential for pathological study , exhuming instead , the skewed melodrama of the circumstantial situation .

[4]:a visually flashy but narratively opaque and emotionally vapid exercise in style and mystification .

Positive reivews:
[0]:the rock is destined to be the 21st century's new " conan " and that he's going to make a splash even greater than arnold schwarzenegger , jean-claud van damme or stev
en segal .

[1]:the gorgeously elaborate continuation of " the lord of the rings " trilogy is so huge that a column of words cannot adequately describe co-writer/director peter jackso
n's expanded vision of j . r . r . tolkien's middle-earth .

[2]:effective but too-tepid biopic

[3]:if you sometimes like to go to the movies to have fun , wasabi is a good place to start .

[4]:emerges as something rare , an issue movie that's so honest and keenly observed that it doesn't feel like one .
```

Code for defining the data processing function:

```
# Define the data generation class.
class Generator():
    def __init__(self, input_list):
        self.input_list=input_list
    def __getitem__(self,item):
        return (np.array(self.input_list[item][0],dtype=np.int32),
                np.array(self.input_list[item][1],dtype=np.int32))
    def __len__(self):
        return len(self.input_list)


class MovieReview:
```

```
    '''
    Movie review dataset
    '''
    def __init__(self, root_dir, maxlen, split):
        '''
        input:
            root_dir: movie review data directory
            maxlen: maximum length of a sentence
            split: the training/validation ratio in the dataset
        '''
        self.path = root_dir
        self.feelMap = {
            'neg':0,
            'pos':1
        }
        self.files = []

        self.doConvert = False

        mypath = Path(self.path)
        if not mypath.exists() or not mypath.is_dir():
            print("please check the root_dir!")
            raise ValueError

        # Find the files in the data directory.




        # Check whether there are two files: .neg and .pos.




       # Read data.
        self.word_num = 0
        self.maxlen = 0
        self.minlen = float("inf")
        self.maxlen = float("-inf")
        self.Pos = []
        self.Neg = []
        for filename in self.files:
            self.read_data(filename)

        self.text2vec(maxlen=maxlen)
        self.split_dataset(split=split)

    def read_data(self, filePath):
        with open(filePath,'r') as f:
            for sentence in f.readlines():
                sentence = sentence.replace('\n','')\
                                    .replace('"','')\
                                    .replace('\','')\
                                    .replace('.','')\
                                    .replace(',','')\
```

```
                                        .replace('[','')\
                                        .replace(']','')\
                                        .replace('(','')\
                                        .replace(')','')\
                                        .replace(':','')\
                                        .replace('--','')\
                                        .replace('-',' ')\
                                        .replace('\\\','')\
                                        .replace('0','')\
                                        .replace('1','')\
                                        .replace('2','')\
                                        .replace('3','')\
                                        .replace('4','')\
                                        .replace('5','')\
                                        .replace('6','')\
                                        .replace('7','')\
                                        .replace('8','')\
                                        .replace('9','')\
                                        .replace('`','')\
                                        .replace('=','')\
                                        .replace('$','')\
                                        .replace('/','')\
                                        .replace('*','')\
                                        .replace(';','')\
                                        .replace('<b>','')\
                                        .replace('%','')


                sentence = sentence.split(' ')
                sentence = list(filter(lambda x: x, sentence))
                if sentence:
                    self.word_num += len(sentence)
                    self.maxlen = self.maxlen if self.maxlen >= len(sentence) else len(sentence)
                    self.minlen = self.minlen if self.minlen <= len(sentence) else len(sentence)
                    if 'pos' in filePath:
                        self.Pos.append([sentence,self.feelMap['pos']])
                    else:
                        self.Neg.append([sentence,self.feelMap['neg']])

    def text2vec(self, maxlen):
        '''
# Convert sentences into vectors.
        '''
        # Vocab = {word : index}


        # self.Vocab['None']
        for SentenceLabel in self.Pos+self.Neg:
            vector = [0]*maxlen
            for index, word in enumerate(SentenceLabel[0]):
                if index >= maxlen:
                    break
                if word not in self.Vocab.keys():
```

```
                    else:
                        vector[index] = self.Vocab[word]
                SentenceLabel[0] = vector
            self.doConvert = True

    def split_dataset(self, split):
    '''
    Divide the dataset into a training set and a test set.
    '''
            trunk_pos_size = math.ceil((1-split)*len(self.Pos))
            trunk_neg_size = math.ceil((1-split)*len(self.Neg))
            trunk_num = int(1/(1-split))
            pos_temp=list()
            neg_temp=list()
            for index in range(trunk_num):
                pos_temp.append(self.Pos[index*trunk_pos_size:(index+1)*trunk_pos_size])
                neg_temp.append(self.Neg[index*trunk_neg_size:(index+1)*trunk_neg_size])
            self.test = pos_temp.pop(2)+neg_temp.pop(2)
            self.train = [i for item in pos_temp+neg_temp for i in item]

    random.shuffle(self.train)

        def get_dict_len(self):
            '''
            Obtain the length of a dictionary consisting of characters in a dataset.
            '''
            if self.doConvert:
                return len(self.Vocab)
            else:
                print("Haven't finished Text2Vec")
                return -1

        def create_train_dataset(self, epoch_size, batch_size):
            dataset = ds.GeneratorDataset(
                                        source=Generator(input_list=self.train),
                                        column_names=["data","label"],
                                        shuffle=False
                                        )
            dataset=dataset.batch(batch_size=batch_size,drop_remainder=True)
            dataset=dataset.repeat(epoch_size)
            return dataset

        def create_test_dataset(self, batch_size):
            dataset = ds.GeneratorDataset(
                                        source=Generator(input_list=self.test),
                                        column_names=["data","label"],
                                        shuffle=False
                                        )
            dataset=dataset.batch(batch_size=batch_size,drop_remainder=True)
            return dataset
```

Use the customized read_data function to load the original dataset and use the text2vec function to vectorize the text of the data. Use split_dataset to split data into training data

and test data, and then create_test_dataset and create_train_dataset call mindspore.dataset. GeneratorDataset generates data for training and validation.

Configuration code:

```
instance = MovieReview(root_dir=cfg.data_path, maxlen=cfg.word_len, split=0.9)
dataset = instance.create_train_dataset(batch_size=cfg.batch_size,epoch_size=cfg.epoch_size)
batch_num = dataset.get_dataset_size()
```

Code for displaying the data processing result:

```
vocab_size=instance.get_dict_len()
print("vocab_size:{0}".format(vocab_size))
item =dataset.create_dict_iterator()
for i,data in enumerate(item):
    if i<1:
        print(data)
        print(data['data'][1])
    else:
        break
```

Output the data processing result:

```
vocab_size:18848
{'data': Tensor(shape=[64, 51], dtype=Int32, value=
[[  15, 3514,   10 ...    0,    0,    0],
 [   0,  889, 1305 ...    0,    0,    0],
 [ 128,   15, 1836 ...    0,    0,    0],
 ...
 [3429, 2678,  203 ...    0,    0,    0],
 [   0,   77,    2 ...    0,    0,    0],
 [5277, 5278,  997 ...    0,    0,    0]]), 'label': Tensor(shape=[64], dtype=Int32, value= [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])}
[   0  889 1305 5099   10   88 2409 1755 1588  186  672    0 5100   32
    0   86 5101   14   82 1059  147    0 1024    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0]
```

Step 3 Configure training parameters.

Code for setting the learning rate:

```
learning_rate = []
warm_up = [1e-3 / math.floor(cfg.epoch_size / 5) * (i + 1) for _ in range(batch_num)
                for i in range(math.floor(cfg.epoch_size / 5))]
shrink = [1e-3 / (16 * (i + 1)) for _ in range(batch_num)
               for i in range(math.floor(cfg.epoch_size * 3 / 5))]
normal_run = [1e-3 for _ in range(batch_num) for i in
                    range(cfg.epoch_size - math.floor(cfg.epoch_size / 5)
                            - math.floor(cfg.epoch_size * 2 / 5))]
learning_rate = learning_rate + warm_up + normal_run + shrink
```

The dynamic learning rate in training is implemented through epoch.

Step 4 Define a TextCNN model.

Model classes define model structure build, training, validation, offline model loading, and online inference functions.

Code:

```
def _weight_variable(shape, factor=0.01):
    init_value = np.random.randn(*shape).astype(np.float32) * factor
    return Tensor(init_value)

def make_conv_layer(kernel_size):
    weight_shape = (96, 1, *kernel_size)
    weight = _weight_variable(weight_shape)
    return nn.Conv2d(in_channels=1, out_channels=96, kernel_size=kernel_size, padding=1,
                     pad_mode="pad", weight_init=weight, has_bias=True)

class TextCNN(nn.Cell):
    def __init__(self, vocab_len, word_len, num_classes, vec_length):
        super(TextCNN, self).__init__()
        self.vec_length = vec_length
        self.word_len = word_len
        self.num_classes = num_classes

        self.unsqueeze = ops.ExpandDims()
        self.embedding = nn.Embedding(vocab_len, self.vec_length, embedding_table='normal')

        self.slice = ops.Slice()
        self.layer1 = self.make_layer(kernel_height=3)
        self.layer2 = self.make_layer(kernel_height=4)
        self.layer3 = self.make_layer(kernel_height=5)

        self.concat = ops.Concat(1)

        self.fc = nn.Dense(96*3, self.num_classes)
        self.drop = nn.Dropout(keep_prob=0.5)
        self.print = ops.Print()
        self.reducemean = ops.ReduceMax(keep_dims=False)

    def make_layer(self, kernel_height):
        return nn.SequentialCell(
            [
                make_conv_layer((kernel_height,self.vec_length)),
                nn.ReLU(),
                nn.MaxPool2d(kernel_size=(self.word_len-kernel_height+1,1)),
            ]
        )

    def construct(self,x):
```

```
net = TextCNN(vocab_len=instance.get_dict_len(), word_len=cfg.word_len,
              num_classes=cfg.num_classes, vec_length=cfg.vec_length)
print(net)
```

View the neural network structure.

```
TextCNN<
  (embedding): Embedding<vocab_size=18848, embedding_size=40, use_one_hot=False, embedding_table=Parameter (name=embedding.embedding_table), dtype=Float32, padding_idx=Non
e>
  (layer1): SequentialCell<
    (0): Conv2d<input_channels=1, output_channels=96, kernel_size=(3, 40),stride=(1, 1),  pad_mode=pad, padding=1, dilation=(1, 1), group=1, has_bias=Trueweight_init=[[[[
1.47466036e-02 -8.85174982e-03  6.20904262e-04 ...  5.97969582e-03
         6.87394897e-03 -4.35171695e-03]
       [ 1.25721507e-02  1.11018270e-02  3.73373111e-03 ... -1.32954121e-02
         1.67019237e-02  6.70977589e-03]
       [ 5.16406354e-03  6.65343972e-03 -9.12646390e-03 ...  4.12355090e-04
         6.96751568e-03  2.65645944e-02]]]


     [[[ 2.20983545e-03  9.51934140e-03  5.41285099e-03 ... -2.79155909e-03
        -9.01016127e-03  1.36778364e-02]
       [-9.36647598e-03 -1.73871801e-03  1.08188950e-02 ...  1.51296274e-03
         7.57558912e-04  5.44300769e-03]
       [ 1.13182161e-02  4.26522718e-04  1.49796065e-03 ... -3.42557859e-03
        -1.79516233e-03 -4.79884632e-03]]]


     [[[-4.98654880e-03 -2.08284725e-02  2.18074489e-02 ... -7.44788814e-03
        -4.96819383e-03  1.28578511e-03]
       [ 1.82666897e-03  1.37834558e-02 -4.40165540e-03 ... -8.17192066e-03
         9.86121874e-03  1.46957850e-02]
       [ 1.46543132e-02  7.15820771e-03  5.12730749e-03 ... -1.95071772e-02
        -1.01376360e-03  6.32047653e-03]]]
```

Step 5 Define training parameters.

A loss function is also called an objective function and is used to measure the difference between a predicted value and an actual value. Deep learning reduces the loss value by continuous iteration. Defining a good loss function can effectively improve model performance.

An optimizer is used to minimize the loss function, improving the model during training.

After the loss function is defined, the weight-related gradient of the loss function can be obtained. The gradient is used to indicate the weight optimization direction for the optimizer, improving model performance. Loss functions supported by MindSpore include SoftmaxCrossEntropyWithLogits, L1Loss, and MSELoss. SoftmaxCrossEntropyWithLogits is used in this example.

MindSpore provides the callback mechanism to execute custom logic during training. The following uses ModelCheckpoint provided by the framework as an example. ModelCheckpoint can save the network model and parameters for subsequent fine-tuning.

Code:

```
# Optimizer, loss function, checkpoint, and time monitor settings
opt =


loss =


model =
```

```
config_ck =


time_cb = TimeMonitor(data_size=batch_num)
ckpt_save_dir = "./ckpt"
ckpoint_cb = ModelCheckpoint(prefix="train_textcnn", directory=ckpt_save_dir, config=config_ck)
loss_cb = LossMonitor()
```

Step 6 Start training.

The training process refers to a process in which training dataset is transferred to a network for training and optimizing network parameters. In the MindSpore framework, the Model.train method is used to complete this process.

Code:

```
model.train(cfg.epoch_size, dataset, callbacks=[time_cb, ckpoint_cb, loss_cb])
print("train success")
```

Output:

```
epoch: 1 step: 596, loss is 0.023292765
epoch time: 36818.071 ms, per step time: 61.775 ms
epoch: 2 step: 596, loss is 0.0016317479
epoch time: 4320.621 ms, per step time: 7.249 ms
epoch: 3 step: 596, loss is 0.00025401893
epoch time: 4271.667 ms, per step time: 7.167 ms
epoch: 4 step: 596, loss is 0.0001774891
epoch time: 4332.598 ms, per step time: 7.269 ms
train success
```

Step 7 Test and validate data.

A single piece of review text data is obtained for testing, and a sentiment category and a probability of the sentiment category are output.

Code:

```
def preprocess(sentence):
    sentence = sentence.lower().strip()
    sentence = sentence.replace('\n','')\
                                .replace('"','')\
                                .replace('\','')\
                                .replace('.','')\
                                .replace(',','')\
                                .replace('[','')\
                                .replace(']','')\
                                .replace('(','')\
                                .replace(')','')\
                                .replace(':','')\
                                .replace('--','')\
                                .replace('-',' ')\
                                .replace('\\','')\
                                .replace('0','')\
                                .replace('1','')\
```

```
                                        .replace('2','')\
                                        .replace('3','')\
                                        .replace('4','')\
                                        .replace('5','')\
                                        .replace('6','')\
                                        .replace('7','')\
                                        .replace('8','')\
                                        .replace('9','')\
                                        .replace('`','')\
                                        .replace('=','')\
                                        .replace('$','')\
                                        .replace('/','')\
                                        .replace('*','')\
            .replace(';','')\
                                        .replace('<b>','')\
                                        .replace('%','')\
                                        .replace("    "," ")
    sentence = sentence.split(' ')
    maxlen = cfg.word_len
    vector = [0]*maxlen
    for index, word in enumerate(sentence):
        if index >= maxlen:
            break
        if word not in instance.Vocab.keys():
            print(word,"The word does not appear in the dictionary.")
        else:
            vector[index] = instance.Vocab[word]
    sentence = vector

    return sentence

def inference(review_en):
    review_en = preprocess(review_en)
    input_en = Tensor(np.array([review_en]).astype(np.int32))
    output = net(input_en)
    if np.argmax(np.array(output[0])) == 1:
        print("Positive comments")
    else:
        print("Negative comments")
```

Code:

```
review_en = "the movie is so boring"
inference(review_en)
```

Output:

```
Negative comments
```

1.4 Question

If you want to perform the preceding exercise on a GPU, which configuration item do you need to change to GPU?